Thrust Vector Control Actuation for the Ranger VTVL Vehicle

Portfolio sample, Summer 2027 Electrical Systems Engineering Internship

Adit Bansal
USC, B.S. Electrical & Computer Eng. 2028
aditbans@usc.edu

1. Overview

The USC Liquid Propulsion Lab is building Ranger, a liquid-fueled vertical takeoff, vertical landing vehicle. The engine is mounted on a two-axis gimbal, and two electric linear actuators push it in pitch and yaw. Since January 2026 I have been on the GNC team working on the thrust vector control (TVC) system between the flight computer and those actuators: the CAN-FD link to the two motor controllers, the 50 Hz control thread, the gimbal kinematics, and the supervisor logic that decides when the actuators are allowed to move.

What I did

- Brought up CAN-FD on the Teensy 4.1's FlexCAN3 controller (devicetree, Kconfig, bit rates, pin assignment)
- Wrote the moteus register protocol encoder/decoder and the CAN transport
- Wrote the 50 Hz TVC thread and fixed timing races between it and the main controller
- Implemented the inverse kinematics from the team's MATLAB gimbal model
- Wrote the safety supervisor: homing, enable checks, limits and fault handling
- Debugged the bus with an oscilloscope and SocketCAN

Teammates wrote the flight computer's controller framework and sensor drivers and ran the first moteus bench test. The gimbal mechanism and its MATLAB model came from the mechanical side.

System parameters

Gimbal range	±12° pitch and yaw
Actuators	BLDC + 4 mm lead screw, 2.4 in stroke
Motor controllers	2× moteus r4.1
Bus	CAN-FD, 1 / 5 Mbps
Flight computer	Teensy 4.1, Zephyr RTOS, C++
Loop rate	50 Hz, replies within 5 ms
Testing	55 unit tests, bench builds

2. System architecture

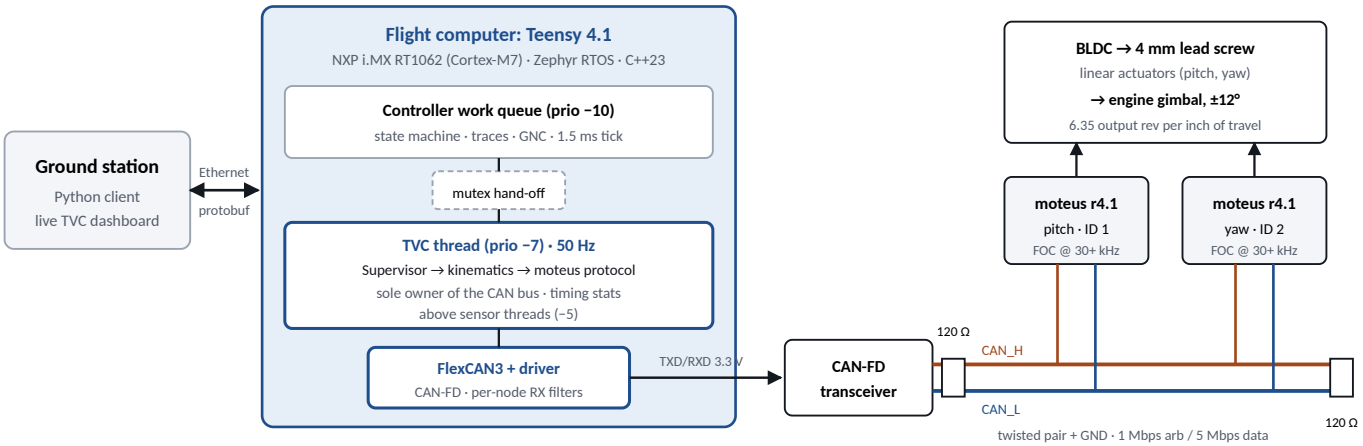


Figure 1. TVC signal path. The blocks outlined in blue are the parts I wrote. Each moteus controller closes its own current and position loops; every 20 ms my firmware sends it a position setpoint with limits and reads back its state.

Requirements

ID	Requirement
R1	Point the engine ±12° in pitch and yaw, matching the mechanical team's kinematic model
R2	Update the actuators every 20 ms, with both replies back within 5 ms
R3	Never drive an actuator that is unhomed, off center at enable, or reporting a fault
R4	Go to a safe state on loss of communication or a motor controller fault
R5	Reject invalid commands (NaN, out of range, large steps)

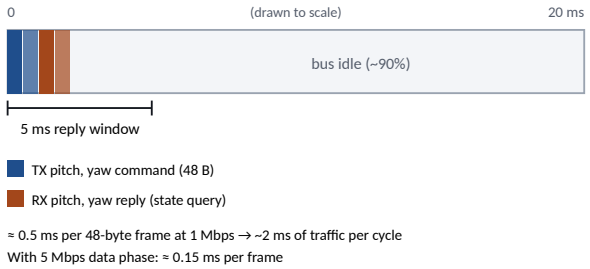
3. CAN-FD interface

Each actuator needs a 43-byte command and status query every cycle. On classic CAN (8 data bytes per frame) that would take six frames per actuator. CAN-FD fits it in one frame of up to 64 bytes and can run the data phase faster. It is also a differential bus, which matters next to a running engine; the PWM servo links on our smaller Hornet test vehicle are single-ended.

Physical layer decisions

Decision	Reason
Transceiver with 3.3 V logic I/O	Teensy 4.x pins are not 5 V tolerant
120 Ω at both ends of the bus	The moteus boards have no termination. An unterminated end reflects edges back onto the bus.
Twisted pair plus GND	Common-mode noise cancels at the receiver; GND keeps both nodes in common-mode range
Transceiver standby pin tied to normal mode	If it floats, the transceiver may not drive the bus or ACK
Start with BRS off (1 Mbps data)	Already proven on the bench; move to 5 Mbps after checking signal quality on the vehicle harness

Bus timing in one 20 ms cycle



Rough frame time without bit rate switching, for a 48-byte frame (43 bytes padded to the next CAN-FD size):

```
t ≈ 67 bits / R_arb + ~430 bits / R_data ≈ 0.5 ms at 1 Mbps
```

Four frames per cycle (two commands, two replies) come to about 2 ms, which fits in the 5 ms reply window with room for a retry. I send register values as float32 instead of moteus' scaled int16 format. The int16 position format tops out at ±3.28 revolutions, and the actuator moves about ±8 revolutions over ±12°, so int16 would overflow.

Firmware side of the interface

- A devicetree overlay enables FlexCAN3 on pins 31 (TX) and 30 (RX) and sets both bit rates. The build fails if CAN-FD is not configured.
- Each motor controller gets its own hardware receive filter on the full 29-bit ID and its own message queue.
- Transmits never block. A frame goes into a mailbox with a completion callback, so a missing ACK can't stall the control thread.
- Each command carries a 100 ms watchdog. If commands stop arriving (cut cable, hung MCU), the moteus stops driving on its own.

Bring-up and debugging

I brought the bus up one actuator at a time using a single-axis bench build, so any failure had one place to look. On the oscilloscope I checked the differential levels on CAN_H and CAN_L (about 0 V recessive, about 2 V dominant) and looked for ringing on the edges, which usually means a termination problem. With power off, the resistance across H and L should be about 60 Ω (two 120 Ω terminators in parallel). SocketCAN (`candump`) let me check IDs and ACKs from a PC without trusting the Teensy. On the firmware side I matched the 29-bit ID layout and bit rates to the moteus configuration.

Interface summary

Signal	From / to	Physical layer
CAN TX, RX	Teensy pins 31 / 30 to transceiver TXD / RXD	3.3 V logic
CAN_H, CAN_L, GND	Transceiver to both moteus (JST PH-3)	Twisted pair, 120 Ω at each end, 1 / 5 Mbps
Commands, telemetry	Teensy to ground station	Ethernet, protobuf messages

4. Control chain and safety logic

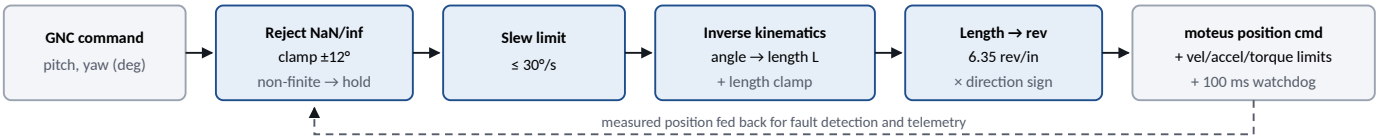
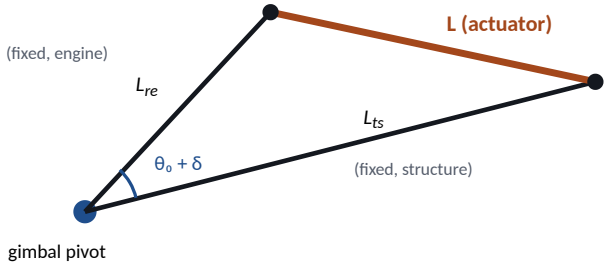


Figure 2. Path from a gimbal angle command to a motor position command, run every 20 ms.

Inverse kinematics

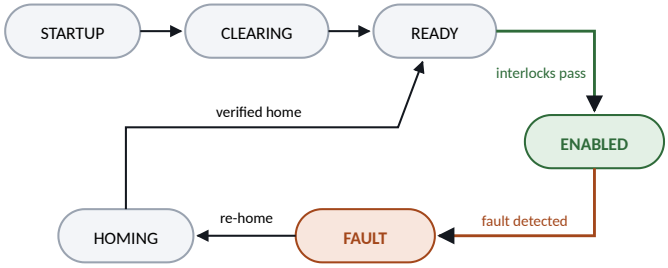


The actuator, the fixed structure arm and the fixed engine arm form a triangle, so the actuator length follows from the law of cosines:

$$L^2 = L_{ts}^2 + L_{re}^2 - 2 \cdot L_{ts} \cdot L_{re} \cdot \cos(\theta_0 + \delta)$$

δ is the gimbal angle and θ_0 is the angle at center. Running it backwards turns the measured motor position into an angle for telemetry and fault checks. The lead screw gives 6.35 motor revolutions per inch, so ±12° is roughly ±8 revolutions. I checked the firmware against 241 points per axis (−12° to +12°) generated from the team's MATLAB model.

Supervisor state machine



The supervisor only enables the actuators when both controllers are replying with no faults, both have been homed since power-up, both are within 0.05 in of center, and none of the configuration values are still placeholders.

Fault	Trigger
No reply	3 missed cycles in a row (60 ms)
Controller fault	moteus reports a fault code
Watchdog timeout	moteus watchdog expired
Wrong mode	Not in position mode while enabled
Tracking error	More than 0.10 in off for 5 cycles

Faults latch until the actuators are re-homed. On a fault, healthy axes return to center, the faulted axis is stopped, and any active static fire sequence aborts.

There are three separate protections against the gimbal moving when it shouldn't: the supervisor's enable checks, the 100 ms watchdog inside each motor controller, and hard limits on angle (±12°), rate (30°/s) and motor current.

5. Timing

The TVC loop runs in its own thread, which is the only code that touches the CAN bus. It waits on a periodic kernel timer instead of sleeping after each cycle. Sleeping after the work makes the period 20 ms plus however long the work took, so the loop slowly drifts; with a periodic timer the period stays at 20 ms, and a missed cycle is counted as an overrun.

The main controller runs on a 1.5 ms work queue. The TVC thread runs at a lower priority than the controller and a higher one than the sensor threads, so a slow sensor read can't delay an actuator command. The two share one small struct behind a mutex, and neither blocks or logs while holding it, so the controller never waits on CAN. CPU time is tens of microseconds per cycle. Every cycle logs period jitter, per-axis reply latency and overruns to the ground station.

Item	Value
Reply window for both actuators	5 ms
Bus traffic per cycle at 1 Mbps	~2 ms (about 10% bus load)
Time to declare a silent actuator faulted	60 ms (3 cycles)
moteus watchdog	100 ms

6. Verification

The supervisor, kinematics and protocol code don't depend on Zephyr or the CAN driver, so I could test them on a PC (Zephyr native_sim) before anything moved. There are 55 unit tests. On hardware, a single-actuator build runs one axis on a bare Teensy with a transceiver, and a bench sweep moves it ±2° at 0.5°/s while logging commanded and measured angle.

Req.	How it is verified
R1	Kinematics unit tests against 241 MATLAB points per axis; bench sweep of commanded vs. measured angle
R2	Per-cycle jitter, latency and overrun telemetry
R3	Unit tests that unhomed, off-center, silent and misconfigured actuators never enable
R4	A unit test for each of the 5 fault types; moteus watchdog as a hardware backup
R5	Unit tests for NaN commands (hold last angle), angle and length clamps, and slew limiting of steps

7. Problems I ran into

Static initialization order. While reviewing the code before bench testing, I found that one geometry constant was computed at program start, while the supervisor used it in its own global constructor. Depending on link order the constant could still be 0 at that point, which would collapse the length limits and pin every command to one length. The unit tests passed because they happened to initialize in the right order. I made the constant compile-time (`constexpr`, with a compile-time square root) and added a `static_assert` on its value, so a geometry change that breaks it fails the build.

Waiting on CAN in the controller. Waiting for CAN replies (up to 5 ms) inside the 1.5 ms controller tick would stall the controller, and the controller and CAN code were racing on shared state. I moved CAN into its own thread with the timer-based loop and mutex hand-off described above. I also moved a log call out of the locked section, because it could stall the controller, and fixed an undefined NaN-to-int conversion in the bench log. Replies now come back within 5 ms at a steady 50 Hz.

8. Next steps and takeaways

Before vehicle integration: check signal quality on the full vehicle harness and turn on the 5 Mbps data phase, set an explicit torque limit from the actuator's rated force, tune the slew and tracking limits from measured step responses, and run hardware-in-the-loop tests with GNC.

The main thing I took from this project is how much of the work was at the hardware boundary: pin voltage limits, termination, bus timing, and what the motor controller does on its own when commands stop. I also learned to keep the logic testable away from the hardware, because it let me check every fault path before an actuator was connected.